RFM Analysis with SQL

```
-- Inspecting the dataset structure (21 columns; preview shown in 3 screenshots)
SELECT * FROM RETAIL_SALES;
```

	ROW_ID	ORDER_ID	ORDER_DATE	SHIP_DATE	SHIP_MODE	CUSTOMER_ID	CUSTOMER_NAME
1	1	CA-2016-152156	2016-11-08 00:00:00.000	2016-11-11 00:00:00.000	Second Class	CG-12520	Claire Gute
2	2	CA-2016-152156	2016-11-08 00:00:00.000	2016-11-11 00:00:00.000	Second Class	CG-12520	Claire Gute
3	3	CA-2016-138688	2016-06-12 00:00:00.000	2016-06-16 00:00:00.000	Second Class	DV-13045	Darrin Van Huff
4	4	US-2015-108966	2015-10-11 00:00:00.000	2015-10-18 00:00:00.000	Standard Class	SO-20335	Sean O'Donnell
5	5	US-2015-108966	2015-10-11 00:00:00.000	2015-10-18 00:00:00.000	Standard Class	SO-20335	Sean O'Donnell
6	6	CA-2014-115812	2014-06-09 00:00:00.000	2014-06-14 00:00:00.000	Standard Class	BH-11710	Brosina Hoffman
7	7	CA-2014-115812	2014-06-09 00:00:00.000	2014-06-14 00:00:00.000	Standard Class	BH-11710	Brosina Hoffman
8	8	CA-2014-115812	2014-06-09 00:00:00.000	2014-06-14 00:00:00.000	Standard Class	BH-11710	Brosina Hoffman
9	9	CA-2014-115812	2014-06-09 00:00:00.000	2014-06-14 00:00:00.000	Standard Class	BH-11710	Brosina Hoffman
10	10	CA-2014-115812	2014-06-09 00:00:00.000	2014-06-14 00:00:00.000	Standard Class	BH-11710	Brosina Hoffman

	SEGMENT	COUNTRY	CITY	STATE	REGION	PRODUCT_ID	CATEGORY	SUB_CATEGORY
1	Consumer	United States	Henderson	Kentucky	South	FUR-BO-10001798	Furniture	Bookcases
2	Consumer	United States	Henderson	Kentucky	South	FUR-CH-10000454	Furniture	Chairs
3	Corporate	United States	Los Angeles	California	West	OFF-LA-10000240	Office Supplies	Labels
4	Consumer	United States	Fort Lauderdale	Florida	South	FUR-TA-10000577	Furniture	Tables
5	Consumer	United States	Fort Lauderdale	Florida	South	OFF-ST-10000760	Office Supplies	Storage
6	Consumer	United States	Los Angeles	California	West	FUR-FU-10001487	Furniture	Furnishings
7	Consumer	United States	Los Angeles	California	West	OFF-AR-10002833	Office Supplies	Art
8	Consumer	United States	Los Angeles	California	West	TEC-PH-10002275	Technology	Phones
9	Consumer	United States	Los Angeles	California	West	OFF-BI-10003910	Office Supplies	Binders
10	Consumer	United States	Los Angeles	California	West	OFF-AP-10002892	Office Supplies	Appliances

	PRODUCT_NAME	SALES	QUANTITY	DISCOUNT	PROFIT	POSTAL_CODE
1	Bush Somerset Collection Bookcase	261.96	2	0	41.9136	42420
2	Hon Deluxe Fabric Upholstered Stack	731.94	3	0	219.582	42420
3	Self-Adhesive Address Labels for Typ	14.62	2	0	6.8714	90036
4	Bretford CR4500 Series Slim Rectang	957.5775	5	0.45	-383.031	33311
5	Eldon Fold 'N Roll Cart System	22.368	2	0.2	2.5164	33311
6	Eldon Expressions Wood and Plastic	48.86	7	0	14.1694	90032
7	Newell 322	7.28	4	0	1.9656	90032
8	Mitel 5320 IP Phone VoIP phone	907.152	6	0.2	90.7152	90032
9	DXL Angle-View Binders with Locking	18.504	3	0.2	5.7825	90032
10	Belkin F5C206VTEL 6 Outlet Surge	114.9	5	0	34.47	90032

```
-- Identify the minimum and maximum order dates in the dataset
SELECT
      MIN(order_date) min_date,
      MAX(order_date) max_date
FROM RETAIL_SALES
```

	MIN_DATE	MAX_DATE
1	2014-01-03 00:00:00.000	2017-12-30 00:00:00.000

```
-- Since the dataset contains historical dates, we use the latest order date (max_date) as the
reference date for analysis:

WITH max_date_table AS (
       SELECT
              max(order_date) AS  max_date
       FROM RETAIL_SALES
),
-- Now starting the RFM calculation
RFM AS(
       SELECT              customer_id,
max_date - MAX(order_date) AS recency,
 COUNT(DISTINCT(order_id)) AS frequency,
              SUM(sales) AS monetary
       FROM RETAIL_SALES r
       CROSS JOIN max_date m
GROUP BY r.customer_id, m.max_date
),
```

	CUSTOMER_ID	RECENCY	FREQUENCY	MONETARY
1	NB-18580	1,165	2	273.872
2	GR-14560	1,135	2	1,284.38
3	RE-19405	1,097	1	48.36
4	CM-12715	1,035	4	3,984.4524
5	VT-21700	1,000	2	1,736.596
6	PC-19000	882	2	1,061.488
7	AG-10525	845	4	1,455.038
8	PF-19120	835	4	9,062.864
9	DP-13165	812	2	1,058.616
10	CC-12685	799	4	2,854.115

```
-- Assign scores from 1 to 5 for RFM

RFM_score AS(
       SELECT
customer_id,
recency,     frequency,
monetary,   NTILE(5)
OVER(ORDER BY recency
DESC)    AS r_score,
              NTILE(5) OVER(ORDER BY frequency)             AS f_score,
              NTILE(5) over(ORDER BY monetary)              AS m_score
       FROM RFM
)
```

	CUSTOMER_ID	RECENCY	FREQUENCY	MONETARY	R_SCORE	F_SCORE	M_SCORE
1	NB-18580	1,165	2	273.872	1	1	1
2	GR-14560	1,135	2	1,284.38	1	1	2
3	RE-19405	1,097	1	48.36	1	1	1
4	CM-12715	1,035	4	3,984.4524	1	1	4
5	VT-21700	1,000	2	1,736.596	1	1	2
6	PC-19000	882	2	1,061.488	1	1	2
7	AG-10525	845	4	1,455.038	1	1	2
8	PF-19120	835	4	9,062.864	1	1	5
9	DP-13165	812	2	1,058.616	1	1	2
10	CC-12685	799	4	2,854.115	1	2	4

```
-- Validate score ranges.
-- This section is only for checking score distribution.
-- We will return to the main RFM flow afterward and continue the CTE pipeline.
 recency_range
AS(
      SELECT
            r_score,
           MIN(recency)  AS rmin,
             MAX(recency)     AS rmax
      FROM rfm_score
      GROUP BY r_score
),
-- The range should be calculated separately for Recency, Frequency,
and Monetary frequency_range AS(
      SELECT
            f_score,
             MIN(frequency)   AS fmin,
           MAX(frequency)  AS fmax
      FROM rfm_score
      GROUP BY f_score
), monetary_range as(
      SELECT
m_score,
    MIN(monetary)        AS    mmin,
max(monetary)    AS  mmax        FROM
rfm_score
      GROUP BY m_score
)
SELECT
      r.r_score,
      r.rmin,
      r.rmax,
      f.f_score,
      f.fmin,
      f.fmax,
      m.m_score,
      m.mmin,
      m.mmax
FROM recency_range r
JOIN frequency_range f
      ON r.r_score=f.f_score
JOIN monetary_range m
      ON r.r_score=m.m_score
ORDER BY r.r_score
```

	R_SCORE	RMIN	RMAX	F_SCORE	FMIN	FMAX	M_SCORE	MMIN	MMAX
1	1	225	1,165	1	1	4	1	4.833	959.477
2	2	103	224	2	4	5	2	964.636	1,739.689
3	3	53	102	3	5	7	3	1,744.7	2,754.216
4	4	26	52	4	7	9	4	2,767.219	4,282.94
5	5	0	26	5	9	17	5	4,299.161	25,043.05

```
-- Calculate the average RFM score and the combined RFM value (continuing from the main flow)
 avg_rfm_concat
AS(
      SELECT
customer_id,
recency,
 frequency,
            monetary,
            r_score ||'-'|| f_score ||'-'|| m_score AS R_F_M,
ROUND((r_score + f_score + m_score) / 3, 2) AS AVG_rfm
      FROM rfm_score
)
```

	CUSTOMER_ID	RECENCY	FREQUENCY	MONETARY	R_F_M	AVG_RFM
1	NB-18580	1,165	2	273.872	1-1-1	1
2	GR-14560	1,135	2	1,284.38	1-1-2	1.33
3	RE-19405	1,097	1	48.36	1-1-1	1
4	CM-12715	1,035	4	3,984.4524	1-1-4	2
5	VT-21700	1,000	2	1,736.596	1-1-2	1.33
6	PC-19000	882	2	1,061.488	1-1-2	1.33
7	AG-10525	845	4	1,455.038	1-1-2	1.33
8	PF-19120	835	4	9,062.864	1-1-5	2.33
9	DP-13165	812	2	1,058.616	1-1-2	1.33
10	CC-12685	799	4	2,854.115	1-2-4	2.33

```
-- Creating Low / Mid / High value segments based on Monetary score
-- Note: Columns used in the next layer must be selected in the previous CTE.
-- Therefore, m_score was added to the prior layer.
```

```
SQL Error [904] [42000]: ORA-00904: "M_SCORE": invalid
identifier
```

```
avg_rfm_concat AS(
    SELECT
        customer_id,
        recency,
        frequency,
        monetary,
        m_score,
        r_score ||'-'|| f_score ||'-'|| m_score AS R_F_M,
        ROUND((r_score + f_score + m_score) / 3, 2) AS AVG_rfm
    FROM rfm_score
),
```

```sql
-- Next, Create Low / Mid / High value segments based on the Monetary score
  value_seg
AS(
      SELECT
customer_id,
recency,
frequency,
monetary,     R_F_M,
             AVG_rfm,
             CASE
                   WHEN m_score in(1,2) THEN 'low'
                    WHEN m_score = 3    THEN 'mid'
                   WHEN m_score in(4,5) THEN 'high'
             END AS value_segment
      FROM avg_rfm_concat
)
```

	CUSTOMER_ID	RECENCY	FREQUENCY	MONETARY	R_F_M	AVG_RFM	VALUE_SEGMENT
1	NB-18580	1,165	2	273.872	1-1-1	1	low
2	GR-14560	1,135	2	1,284.38	1-1-2	1.33	low
3	RE-19405	1,097	1	48.36	1-1-1	1	low
4	CM-12715	1,035	4	3,984.4524	1-1-4	2	high
5	VT-21700	1,000	2	1,736.596	1-1-2	1.33	low
6	PC-19000	882	2	1,061.488	1-1-2	1.33	low
7	AG-10525	845	4	1,455.038	1-1-2	1.33	low
8	PF-19120	835	4	9,062.864	1-1-5	2.33	high
9	DP-13165	812	2	1,058.616	1-1-2	1.33	low
10	CC-12685	799	4	2,854.115	1-2-4	2.33	high

```sql
 -- Segment customers based on their value
customer_seg
as(
      SELECT
customer_id,
recency,     frequency,
 monetary,          R_F_M,
AVG_RFM,
value_segment,
      CASE
                   WHEN r_score >=4 AND f_score >=4 AND m_score >=4 THEN 'VIP'
                   WHEN f_score >=3 AND m_score <4 THEN 'Regular'
                   WHEN r_score <=3 AND r_score>1 THEN 'Dormat'
                   WHEN r_score =1 THEN 'Churned'
                   WHEN r_score >=4 AND f_score <=4 THEN 'New Customer'
             END AS cust_seg
      FROM value_seg
)
```

	CUSTOMER_ID	RECENCY	FREQUENCY	MONETARY	R_F_M	AVG_RFM	VALUE_SEGMENT	CUST_SEG
1	NB-18580	1,165	2	273.872	1-1-1	1	low	Churned
2	GR-14560	1,135	2	1,284.38	1-1-2	1.33	low	Churned
3	RE-19405	1,097	1	48.36	1-1-1	1	low	Churned
4	CM-12715	1,035	4	3,984.4524	1-1-4	2	high	Churned
5	VT-21700	1,000	2	1,736.596	1-1-2	1.33	low	Churned
6	PC-19000	882	2	1,061.488	1-1-2	1.33	low	Churned
7	AG-10525	845	4	1,455.038	1-1-2	1.33	low	Churned
8	PF-19120	835	4	9,062.864	1-1-5	2.33	high	Churned
9	DP-13165	812	2	1,058.616	1-1-2	1.33	low	Churned
10	CC-12685	799	4	2,854.115	1-2-4	2.33	high	Churned

```
-- Note: Since r_score, f_score, and m_score are used in the CASE WHEN logic,
-- these columns were included in the previous CTE layers.

-- Next, create a VIEW based on the final output
-- The full version of the SQL logic is provided below


CREATE OR REPLACE VIEW RFM_ANALYSIS AS
WITH max_date_table AS (
      SELECT
            max(order_date) AS  max_date
      FROM RETAIL_SALES
),
RFM AS(
      SELECT
customer_id,
            max_date - MAX(order_date) AS recency,
 COUNT(DISTINCT(order_id)) AS frequency,
            SUM(sales) AS monetary
      FROM RETAIL_SALES r
      CROSS JOIN max_date_table m
GROUP BY r.customer_id, m.max_date
),
RFM_score AS(
      SELECT
customer_id,
recency,
 frequency,
            monetary,
            NTILE(5) OVER(ORDER BY recency DESC)    AS r_score,
            NTILE(5) OVER(ORDER BY frequency)             AS f_score,
            NTILE(5) over(ORDER BY monetary)              AS m_score
      FROM RFM
), avg_rfm_concat
AS(
      SELECT
customer_id,
 recency,
            frequency,
 monetary,          r_score,
      f_score,
 m_score,
            r_score ||'-'|| f_score ||'-'|| m_score AS R_F_M,
ROUND((r_score + f_score + m_score) / 3, 2) AS AVG_rfm
```

```sql
        FROM rfm_score
), value_seg
AS(
        SELECT
customer_id,
recency,
frequency,
monetary,      R_F_M,
  AVG_rfm,
r_score,
f_score,
 m_score,
              CASE
                     WHEN m_score in(1,2) THEN 'low'
                      WHEN m_score = 3    THEN 'mid'
                     WHEN m_score in(4,5) THEN 'high'
              END AS value_segment
        FROM avg_rfm_concat
), customer_seg as(
        SELECT
customer_id,
recency,      frequency,
 monetary,           R_F_M,
AVG_RFM,
value_segment,
              CASE
                     WHEN r_score >=4 AND f_score >=4 AND m_score >=4 THEN 'VIP'
                     WHEN f_score >=3 AND m_score <4 THEN 'Regular'
                     WHEN r_score <=3 AND r_score>1 THEN 'Dormat'
                     WHEN r_score =1 THEN 'Churned'
                     WHEN r_score >=4 AND f_score <=4 THEN 'New Customer'
              END AS cust_seg
        FROM value_seg
)
SELECT * FROM customer_seg
```

Name	Value
Updated Rows	0
Execute time	0.341s
Start time	Mon Feb 23 18:37:02 AZT 2026
Finish time	Mon Feb 23 18:37:02 AZT 2026
	CREATE OR REPLACE VIEW RFM_ANALYSIS AS

RFM_ANALYSIS

Columns

- A-Z CUSTOMER_ID (VARCHAR2(50))
- 123 RECENCY (NUMBER(38,0))
- 123 FREQUENCY (NUMBER)
- 123 MONETARY (NUMBER)
- A-Z R_F_M (VARCHAR2(122))
- 123 AVG_RFM (NUMBER)
- A-Z VALUE_SEGMENT (VARCHAR2(4))
- A-Z CUST_SEG (VARCHAR2(12))

```
-- VIEW successfully created

SELECT * FROM RFM_ANALYSIS
```

	CUSTOMER_ID	RECENCY	FREQUENCY	MONETARY	R_F_M	AVG_RFM	VALUE_SEGMENT	CUST_SEG
1	NB-18580	1,165	2	273.872	1-1-1	1	low	Churned
2	GR-14560	1,135	2	1,284.38	1-1-2	1.33	low	Churned
3	RE-19405	1,097	1	48.36	1-1-1	1	low	Churned
4	CM-12715	1,035	4	3,984.4524	1-1-4	2	high	Churned
5	VT-21700	1,000	2	1,736.596	1-1-2	1.33	low	Churned
6	PC-19000	882	2	1,061.488	1-1-2	1.33	low	Churned
7	AG-10525	845	4	1,455.038	1-1-2	1.33	low	Churned
8	PF-19120	835	4	9,062.864	1-1-5	2.33	high	Churned
9	DP-13165	812	2	1,058.616	1-1-2	1.33	low	Churned
10	CC-12685	799	4	2,854.115	1-2-4	2.33	high	Churned

```
-- Check the number of customers in each value segment

SELECT
       VALUE_SEGMENT ,
       COUNT(CUSTOMER_ID) AS cust_count
FROM RFM_ANALYSIS
GROUP BY VALUE_SEGMENT
```

	VALUE_SEGMENT	CUST_COUNT
1	low	318
2	mid	159
3	high	316

```sql
-- Identify which segment has the highest customer share and which generates the highest total revenue

SELECT
      CUST_SEG,
      COUNT(CUSTOMER_ID) AS cust_count,
      ROUND(SUM(MONETARY)) AS sum_monetary
FROM RFM_ANALYSIS
GROUP BY CUST_SEG
ORDER BY sum_monetary DESC
```

	A-Z CUST_SEG	123 CUST_COUNT	123 SUM_MONETARY
1	Dormat	231	792,601
2	VIP	96	512,476
3	Regular	215	369,691
4	New Customer	118	319,391
5	Churned	133	302,761

```sql
-- Calculate the percentage share of each segment within the total customer base

SELECT
cust_seg,
    COUNT(*) AS cust_count,
    ROUND(COUNT(*) * 100 / SUM(COUNT(*)) OVER()) AS percentage_segments
FROM RFM_ANALYSIS
GROUP BY cust_seg
ORDER BY percentage_segments DESC;
```

	CUST_SEG	CUST_COUNT	PERCENTAGE_SEGMENTS
1	Dormat	231	29
2	Regular	215	27
3	Churned	133	17
4	New Customer	118	15
5	VIP	96	12

```sql
-- Distribution of customer behavior segments within each value tier
 SELECT
value_segment,
cust_seg,
    COUNT(*) AS cust_count
FROM RFM_ANALYSIS
GROUP BY value_segment, cust_seg
ORDER BY value_segment, cust_count DESC;
```

	VALUE_SEGMENT	CUST_SEG	CUST_COUNT
1	high	Dormat	137
2	high	VIP	96
3	high	New Customer	50
4	high	Churned	33
5	low	Regular	111
6	low	Churned	81
7	low	Dormat	70
8	low	New Customer	56
9	mid	Regular	104
10	mid	Dormat	24
11	mid	Churned	19
12	mid	New Customer	12